به نام خدا



دانشکده ریاضی و آمار

اصول سیستم‌های عامل
نیم‌سال اول ۰۴-۰۵
استاد: دکتر رفیعی
نام دانشجو

Problem Set 1

۱. صحیح یا غلط بودن گزاره‌های زیر را تعیین کنید و در صورت غلط بودن، توضیح دهید چرا غلط است.

(آ) قالب داده و محل قرارگیری آن در ناحیه حافظه اشتراکی به طور مستقیم توسط سیستم عامل کنترل و مدیریت می‌شود.

(ب) به طور معمول، ناحیه حافظه اشتراکی در فضای آدرس فرآیندی که ارتباط را آغاز می‌کند (فرآیند ارتباطی) ایجاد می‌شود.

(ج) در مدل «بافر نامحدود» (Unbounded Buffer)، فرآیند مصرف‌کننده هرگز در حالت انتظار قرار نمی‌گیرد.

(د) در آدرس‌دهی مستقیم «نامتقارن» (Asymmetric) ، گیرنده برای دریافت پیام باید نام فرستنده را بداند.

(ه) در بافر با «ظرفیت صفر» (سیستم پیام بدون بافر)، فرستنده باید تا زمانی که گیرنده پیام را دریافت کند، بلاک شود.

(و) وب‌سرورها برای مدیریت درخواست‌های هم‌زمان کاربران، معمولاً برای هر درخواست یک «فرآیند» جدید ایجاد می‌کنند زیرا سربار کمتری دارد.

(ز) در مدل «چند به یک» (Many-to-One)، اگر یکی از نخ‌های سطح کاربر یک فراخوان سیستمی مسدودکننده (Blocking System Call) انجام دهد، کل فرآیند بلاک می‌شود.

(ح) مدیریت نخ‌های سطح کاربر (User Threads) نیازمند پشتیبانی مستقیم سیستم‌عامل (کرنل) است.

(ط) در مدل «چند به یک»، نخ‌های سطح کاربر می‌توانند به صورت موازی روی سیستم‌های چند هسته‌ای اجرا شوند.

(ی) در استراتژی «نخ ضمنی» (Implicit Threading)، وظیفه ایجاد و مدیریت نخ‌ها از برنامه‌نویس به سیستم‌عامل (کرنل) منتقل می‌شود.

(ک) در مدل‌های چند به چند، اگر نخ سطح کرنل متصل به یک LWP بلاک شود (مثلاً برای I/O)، نخ سطح کاربر متصل به آن LWP همچنان می‌تواند به اجرا ادامه دهد.

پاسخ:

(آ) غلط. قالب داده و محل آن توسط فرآیندهای همکار کنترل می‌شود، نه سیستم عامل.

(ب) صحیح.

(ج) غلط. حتی در بافر نامحدود، اگر بافر خالی باشد (انتظار برای اضافه شدن عنصر جدید)، مصرف‌کننده ممکن است منتظر بماند.

(د) غلط. در طرح نامتقارن، گیرنده نیازی به نام فرستنده ندارد؛ بلکه می‌تواند از هر شناسه‌ای (id) دریافت کند.

(ه) صحیح.

(و) غلط. معمولاً از نخ‌ها استفاده می‌شود زیرا سربار کمتری دارند.

(ز) صحیح.

(ح) غلط. مدیریت آن‌ها بدون پشتیبانی سیستم‌عامل و در فضای کاربر انجام می‌شود.

(ط) غلط. چون تنها یک نخ کرنل وجود دارد، تنها یک نخ در یک زمان قابل اجراست و امکان اجرای موازی وجود ندارد.

(ی) غلط. این وظیفه به کامپایلرها و کتابخانه‌ها منتقل می‌شود.

(ک) غلط. اگر نخ کرنل بلاک شود، LWP و متعاقباً نخ کاربر متصل به آن نیز بلاک خواهند شد.

۲. (۱۲ نمره) به سوالات زیر پاسخ مناسب دهید.

(آ) ساختار داده مناسب برای پیاده‌سازی بافر محدود (Bounded Buffer) چیست؟

(ب) دو فرآیند برای برقراری ارتباط از طریق حافظه اشتراکی، باید چه محدودیت خاصی که توسط سیستم‌عامل اعمال می‌شود را حذف کنند؟

(ج) فرآیندها چه تضمینی را باید در مورد نوشتن در مکان حافظه اشتراکی ارائه دهند؟

(د) پیوند ارتباطی به چه روش‌هایی می‌تواند به صورت فیزیکی پیاده‌سازی شود؟

(ه) هنگامی که مالکیت صندوق پستی با سیستم عامل است، سیستم عامل چه سه مکانیزمی را باید فراهم کند؟

(و) مزیت «پاسخگویی سریع» در برنامه‌های چندنخی به چه معناست؟

(ز) یک نخ (Thread) به عنوان واحد پایه استفاده از CPU، شامل چه ۴ جزء اختصاصی است؟

(ح) چه منابعی بین تمام نخ‌های یک فرآیند مشترک هستند؟ (۳ مورد نام ببرید)

(ط) چهار مزیت اصلی برنامه‌نویسی چندنخی که در اسلایدها ذکر شده است را نام ببرید.

(ی) پنج چالش اصلی برنامه‌نویسی در سیستم‌های چندهسته‌ای را نام ببرید.

(ک) چرا مدل «یک به یک» هم‌روندی (Concurrency) بیشتری نسبت به مدل «چند به یک» دارد؟

(ل) دو مشکل اصلی ایجاد نخ جدید برای هر درخواست (بدون استفاده از استخر نخ) را نام ببرید.

(م) تفاوت اصلی TLS با داده‌های ایستا (Static Data) چیست؟

پاسخ:

(آ) صف حلقوی (Circular Queue).

(ب) آن‌ها باید توافق کنند که محدودیت سیستم عامل را که در حالت عادی مانع دسترسی یک فرآیند به فضای آدرس فرآیند دیگر می‌شود، حذف کنند.

(ج) ۱۴. فرآیندها باید ضمانت کنند که به طور همزمان روی مکان یکسانی نمی‌نویسند.

(د) ۱۵. از طریق حافظه اشتراکی، گذرگاه سخت‌افزاری، یا شبکه.

(ه) ۱۶. ۱. امکان ایجاد صندوق پستی جدید. ۲. امکان ارسال و دریافت پیام از طریق صندوق پستی. ۳. امکان حذف صندوق پستی.

(و) ۱۷. به این معنی است که اگر بخشی از برنامه بلاک شود، سایر بخش‌ها می‌توانند به اجرا ادامه دهند.

(ز) ۱۸. ۱. شناسه نخ (Thread ID)، ۲. شمارنده برنامه (Program Counter)، ۳. مجموعه ثبات (Register Set)، ۴. پشته (Stack).

(ح) ۱۹. ۱. بخش کد، ۲. بخش داده، ۳. منابع سیستم‌عامل (مانند فایل‌های باز).

(ط) ۲۰. ۱. پاسخگویی سریع، ۲. اشتراک‌گذاری منابع، ۳. صرفه اقتصادی، ۴. مقیاس‌پذیری.

(ی) ۲۱. ۱. شناسایی وظایف، ۲. تعادل، ۳. جداسازی داده، ۴. وابستگی داده، ۵. تست و اشکال‌زدایی.

(ک) ۲۲. زیرا بلاک شدن یک نخ در سطح کاربر باعث بلاک شدن کل فرآیند نمی‌شود و سایر نخ‌ها می‌توانند به کار خود ادامه دهند.

(ل) ۲۳. ۱. سربار زمانی ایجاد نخ و حذف آن پس از اتمام کار. ۲. عدم وجود محدودیت برای تعداد نخ‌های فعال که می‌تواند باعث هدررفت منابع شود.

(م) ۲۴. داده‌های ایستا معمولاً بین همه نخ‌ها مشترک هستند، اما داده‌های TLS برای هر نخ یکتا و اختصاصی هستند.

۳. (۱۴ نمره) در یک سیستم‌عامل ، دو تسک از Mailbox برای ارسال پیام استفاده می‌کنند. وقتی تعداد پیام‌ها زیاد می‌شود، سیستم دچار تأخیر و از دست رفتن Deadline می‌شود. توضیح دهید چرا این اتفاق می‌افتد و چه روش‌هایی برای کاهش این مشکل وجود دارد.

پاسخ:

دلایل رخ‌دادن:

- Mailbox ظرفیت محدود دارد و وقتی پر شود، تسک فرستنده بلوکه می‌شود و زمان پاسخ بالا می‌رود.
- ارسال و دریافت پیام شامل کپی‌کردن داده و Context Switch است که در بار سنگین باعث افزایش تأخیر می‌شود.
- اگر Consumer اولویت پایین داشته باشد، خالی‌شدن Mailbox دیر انجام می‌شود و عملاً Priority Inversion رخ می‌دهد.

راه‌حل‌ها:

- بالا بردن اولویت Consumer.
- ارسال اشاره‌گر به داده به‌جای کپی کامل پیام.
- استفاده از حالت غیرمسدودکننده یا Timeout.
- افزایش ظرفیت Mailbox براساس نرخ واقعی تولید و مصرف.

۴. (۱۲ نمره) در یک برنامه چندنخی، یک Thread Pool با ۸ نخ وجود دارد و سه نوع کار اجرا می‌شود: کارهای سنگین CPU ، کارهای I/O ، و کارهای سبک با اهمیت زمانی. گاهی کارهای سبک دیر اجرا می‌شوند و CPU هم بعضی وقت‌ها بی‌استفاده می‌ماند. توضیح دهید چرا چنین وضعی پیش می‌آید و چه روش‌هایی می‌تواند آن را بهتر کند.

پاسخ:

دلایل رخ‌دادن:

- صف مشترک باعث Head-of-Line Blocking می‌شود و کارهای سبک پشت کارهای سنگین گیر می‌کنند.
- کارهای CPU-bound نخ‌ها را برای مدت زیاد اشغال می‌کنند و باعث اشباع Pool می‌شوند.
- کارهای I/O-bound نخ را هنگام انتظار بلوکه می‌کنند و CPU گاهی بیکار می‌ماند.

راه‌حل‌ها:

- جداکردن صف‌ها و ساختن Thread هایPool مجزا برای انواع کارها.
- استفاده از Work-Stealing برای کاهش انسداد صف.
- استفاده از Priority Queue برای اجرای سریع کارهای سبک و حساس.

۵. (۱۰ نمره) دو برنامه همکار میخواهند پیامی را با استفاده از روش اشتراک حافظه بین یکدیگر انتقال دهند. مشکلات این دو برنامه را یافته و برای هریک راه‌حلی ارائه دهید.

```c
// Progam 1
#include <stdio.h>
#include <unistd.h>
#include <sys/shm.h>
#include <string.h>

int main()
{
    int shmid;
    void *shared_memory;
    char buffer[1024];
    shmid = shmget((key_t)2345, 512, 0666);
    shared_memory = shmat(shmid, NULL, 0);
    printf("Enter some data to write to shared memory\n");
    read(0, buffer, 1024);
    strcpy(shared_memory, buffer);
    shmdt(shared_memory);
    return 0;
}
```

```c
// Progam 2
#include <stdio.h>
#include <sys/shm.h>
#include <string.h>

int main()
{
    int shmid;
    void *shared_memory;
    shmid = shmget((key_t)1234, 512, 0666);
    shared_memory = shmat(shmid, NULL, 0);
    printf("Data read from shared memory: %s\n", shared_memory);
    shmdt(shared_memory);
    return 0;
}
```

پاسخ:

۶. (۲۰ نمره) سه برنامه مستقل می‌خواهند با استفاده از مکانیزم ارسال پیام (Message Passing) با یکدیگر ارتباط برقرار کنند:

- **برنامه A:** اعداد صحیح را از ورودی دریافت کرده و به صف پیام ارسال می‌کند. با ورود عدد 0 برنامه متوقف می‌شود.
- **برنامه B:** فقط اعداد **زوج** را از صف دریافت کرده و مجموع آن‌ها را چاپ می‌کند.
- **برنامه C:** فقط اعداد **فرد** را از صف دریافت کرده و تعداد آن‌ها را چاپ می‌کند.

توضیح دهید چه تغییراتی در برنامه A لازم است و برنامه‌های B و C چگونه باید پیام‌ها را دریافت کنند؟
این کد را تکمیل کنید تا برنامه A به درستی کار کند.

```c
// Program A - Sender
#include <stdio.h>
#include <stdlib.h>
#include <sys/msg.h>

struct Message {
    long int msg_type;
    int number;
};

int main() {
    int msgid = msgget((key_t)5678, 0666 | IPC_CREAT);
    struct Message msg;
    int num;

    while (1) {
        printf("Enter a number (0 to exit): ");
        scanf("%d", &num);
        if (num == 0) break;

        msg.number = num;
        // ...

        msgsnd(msgid, &msg, sizeof(int), 0);
    }
    return 0;
}
```

پاسخ:

۷. (۲۰ نمره) برنامه زیر را در نظر بگیرید و به سوالات پاسخ دهید.

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int counter = 0;

void *thread_func(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 3; i++) {
        counter++;
        printf("Thread %d: counter = %d\n", id, counter);
        sleep(1);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, thread_func, &id1);
    pthread_create(&t2, NULL, thread_func, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter = %d\n", counter);
    return 0;
}
```

(آ) اگر خط `pthread_join(t1, NULL);` را حذف کنیم، چه اتفاقی می‌افتد؟

(ب) تفاوت اصلی بین ایجاد یک نخ جدید با `pthread_create` و ایجاد یک فرآیند جدید با `fork` چیست؟

پاسخ: